

Hàm

Nguyễn Huyền Châu – AI Lab/TLU

Nội dung

- Hàm
- Truyền tham số
- Trả về giá trị
- Các loại biến
- Tham số mặc định
- Overload hàm

Bài toán minh họa

- *Viết chương trình nhập vào 2 số a, b , sau đó hiển thị thực đơn lựa chọn các phép toán: cộng, trừ, nhân, chia và chức năng thoát. Sau khi thực hiện phép toán, chương trình sẽ hiển thị lại thực đơn để người dùng có thể lựa chọn phép toán khác. Quá trình này được lặp lại cho đến khi người dùng chọn thoát*

Hàm

- ❑ Lập trình hướng cấu trúc
- ❑ Định nghĩa hàm
- ❑ Gọi hàm
- ❑ Nguyên mẫu hàm

Lập trình hướng cấu trúc

- “Chia để trị”: chia mã chương trình vào nhiều module nhỏ hơn:
 - thay vì chứa mọi mã, main chứa lời gọi đến các module
 - quản lý nhiều phần nhỏ dễ hơn quản lý một main lớn
 - dễ dùng lại mã (??) và dễ sửa mã (??)
- Module:
 - có thể là hàm (~ học kỳ này) hay lớp (~ học kỳ tới)
 - có thể do người lập trình tạo ra (module mới)
 - hoặc đã có trong thư viện chuẩn (module đóng gói sẵn)

Lập trình hướng cấu trúc

This program has one long, complex function containing all of the statements necessary to solve a problem.

[illegible]

In this program the problem has been divided into smaller problems, each of which is handled by a separate function.

```
int main()  
{  
    statement;  
    statement;  
    statement;  
}
```

main function

```
void function2()
{
    statement;
    statement;
    statement;
}
```

function 2

```
void function3()
{
    statement;
    statement;
    statement;
}
```

function 3

```
void function4()
{
    statement;
    statement;
    statement;
}
```

function 4

Lập trình hướng cấu trúc

Viết chương trình nhập vào 2 số a, b , sau đó hiển thị thực đơn lựa chọn các phép toán: cộng, trừ, nhân, chia và chức năng thoát. Sau khi thực hiện phép toán, chương trình sẽ hiển thị lại thực đơn để người dùng có thể lựa chọn phép toán khác. Quá trình này được lặp lại cho đến khi người dùng chọn thoát

– Phân rã hàm (module hoá)

- Nhập liệu => hàm?
- Chạy menu: => hàm
 - In menu => hàm
 - Đọc lựa chọn => không thành hàm
 - So khớp lựa chọn
 - Cộng => hàm
 - Trừ => hàm
 - Nhân => hàm
 - Chia => hàm
 - Thoát => hàm

Cách định nghĩa hàm

```
<kiểu trả về> <tên hàm> ( <danh sách tham số> ) // tiêu đề hàm  
{  
    <các khai báo biến và câu lệnh> // thân hàm  
}
```

- **Kiểu trả về:** kiểu của giá trị hàm sẽ trả về (không có thì đề kiểu void)
 - chỉ khai báo được một giá trị trả về
- **Tên hàm:** quy tắc giống tên biến, hằng, lớp, ...
- **Danh sách tham số:** tham số chính là các biến để lưu thông tin từ hàm gọi (caller) truyền đến hàm bị gọi (called)
 - Khai báo tương tự biến, nhiều tham số thì tách bởi dấu ,
 <kiểu> <tên tham số 1>, <kiểu> <tên tham số 2>, ...
 - Không có thì để trống hay điền void

Ví dụ: Định nghĩa hàm

```
float square (float y){  
    return y * y;  
}
```

```
int main(){  
    ...  
    cout << square (x);  
    ...  
}
```

- Thân hàm: có thể gồm mọi loại lệnh, cấu trúc như ta vẫn viết với với hàm main
- Lệnh **return**
 - giúp hàm trả về kết quả đã khai báo
 - với hàm khai báo không trả về gì (void), vẫn có thể dùng **return**; để kết thúc hàm
- Hàm dừng khi gặp return hoặc hết lệnh, khi đó điều khiển trả về nơi gọi (caller)
- Không thể định nghĩa một hàm bên trong một hàm khác

Thực hành: Định nghĩa hàm

Giá bán lẻ:

Yêu cầu người dùng nhập giá bán buôn và phần trăm phụ trội chi phí khi bán lẻ của một mặt hàng, từ đó hiển thị giá bán lẻ. Ví dụ:

- Nếu giá bán buôn là 5\$ và phần trăm phụ trội khi bán lẻ là 100%, thì giá bán lẻ là 10\$.
- Nếu giá bán buôn là 5\$ và phần trăm phụ trội khi bán lẻ là 50%, thì giá bán lẻ là 7.5\$.

Viết hàm `calculateRetail` nhận vào giá bán buôn và phần trăm chi phí bán lẻ, trả về giá bán lẻ.

Xác thực: Không chấp nhận chi phí bán buôn hoặc phần trăm chi phí bán lẻ là số âm.

Mỗi tham số được khai báo kiểu, rồi đến tên, các tham số tách nhau bởi phẩy

```
double calculateRetail(double price , double rate)
{
    return price * (1 + rate/100); // return được cả biểu thức
}
```

Sau return là một giá trị cùng kiểu hay ép được sang kiểu trả về

Cách gọi hàm

- Lời gọi hàm:

<tên hàm> (<danh sách đối số>) // đối số là giá trị cho tham số
print(); sqrt(x); sum(m,n); pythagoreCheck(a,b,c); ...

- Số lượng, vị trí, kiểu của đối số khớp với tham số.

- float pow (float a, float b) trả về a^b , để tính 9^{25} thì gọi `cout << pow(9, 25)`

- Kết quả trả về nếu có sẽ dùng được như mọi giá trị bình thường:

```
print();           // không có giá trị trả về thì gọi hàm như một lệnh
sum(m,n);          // có giá trị trả về thì có thể vừa gọi như một lệnh
x = sum(m, n);      // vừa như một giá trị, ở đây là để gán
y = sum(m, n) * 3;  // để tính toán trong biểu thức
z = sum (sqrt(m), sqrt(n)); // để làm đối số cho hàm
if (pythagoreanCheck(a, b, c)) cout << " La 3 cạnh tam giác vuông\n"; // làm điều kiện
cout << pow(x, y) << endl; // để in ra
```

- Một hàm A gọi được một hàm B miễn B định nghĩa trước A

Ví dụ: Định nghĩa và gọi hàm

```
double calculateRetail(double price , double rate)
{
    return price * (1 + rate/100); // return được cả biểu thức
}

int main()
{
    double price, retailRate;
    do{
        cout << "Gia ban buon: ";
        cin >> price;
    }while (price < 0); // còn âm thì còn nhập lại

    do{
        cout << "Ty le chi phi ban le: ";
        cin >> retailRate;
    }while (retailRate < 0); // còn âm thì còn nhập lại

    cout << " Gia ban le: " << calculateRetail (price, retailRate) << endl;
    return 0;
}
```

Thực hành: Định nghĩa và gọi hàm

Tiền tiết kiệm tương lai:

Biết lãi suất ngân hàng theo tháng và số tháng gửi, công thức để tính số tiền tiết kiệm nhận được sau t tháng là $F = P \times (1 + i)^t$ trong đó:

- F là số tiền nhận được trong tương lai
- P số tiền bạn có ban đầu
- i là lãi suất ngân hàng theo tháng
- t là số tháng gửi

Viết hàm `futureValue` với các đầu vào là P , i , t và trả ra giá trị F .

Viết chương trình cho phép người dùng nhập vào P , i , t và in ra màn hình giá trị F .

```
double futureValue (double P, double i, int t)
{
    return P * pow(1+ i/100, t); //nếu i là 20 (%) thì lãi suất là 0.2, nên chia cho 100
}

int main()
{
    double P, i;
    int t;
    cout << "Nhập số tiền ban đầu, lãi ngân hàng, số tháng gửi:";
    cin >> P >> i >> t;
    cout << "Số tiền sẽ nhận được: " << futureValue(P, i, t);
    return 0;
}
```

Thực hành: Định nghĩa và gọi hàm

Thang đo nhiệt độ:

Biết công thức đổi độ F sang độ C như sau: $C = 5/9(F-32)$.

Viết hàm celsius nhận vào độ F và trả về độ C tương ứng.

Rồi viết chương trình minh họa hàm trong đó sử dụng một vòng lặp để hiển thị các giá trị từ 0 – 20 trong thang độ F và giá trị tương ứng trong thang độ C.

```
#include <iostream>
using namespace std;

double celsius(double f)
{
    return 5.0 / 9 * (f - 32); //ép kiểu để có phép chia số thực
}

int main()
{
    for (int i = 0; i <= 20; i++)
        cout << i << " do F = " << celsius (i) << " do C \n";
    return 0;
}
```

Do C =>	do F:
0:	-17.78
1:	-17.22
2:	-16.67
3:	-16.11
4:	-15.56
5:	-15.00
6:	-14.44
7:	-13.89
8:	-13.33
9:	-12.78
10:	-12.22
11:	-11.67
12:	-11.11
13:	-10.56
14:	-10.00
15:	-9.44
16:	-8.89
17:	-8.33
18:	-7.78
19:	-7.22
20:	-6.67

Thực hành: Định nghĩa và gọi hàm

Viết chương trình nhập vào 2 số a, b , sau đó hiển thị thực đơn lựa chọn các phép toán: cộng, trừ, nhân, chia và chức năng thoát. Sau khi thực hiện phép toán, chương trình sẽ hiển thị lại thực đơn để người dùng có thể lựa chọn phép toán khác. Quá trình này được lặp lại cho đến khi người dùng chọn thoát

- Phân rã hàm (module hoá)
- Định nghĩa hàm
- Gọi hàm

- Nhập liệu => làm sao 1 hàm trả về 2 giá trị a và b ? => để sau
- Chạy menu: => hàm chạy menu
 - In menu => hàm in
 - Đọc lựa chọn => vẫn hàm in
 - So khớp lựa chọn
 - Cộng => hàm cộng
 - Trừ => hàm trừ
 - Nhân => hàm nhân
 - Chia => hàm chia
 - Thoát => lệnh

```

int printMenu(){
    cout << "Nhap thao tac ban muon thuc hien: \n"
    << "1. Tinh tong \n"
    << "2. Tinh hieu \n"
    << "3. Tinh tich\n"
    << "4. Tinh thuong\n"
    << "5. Thoat\n";
    int c;
    cin >> c;
    return c;
}

```

```

void executeMenu(double a, double b){
    do {
        int choice = printMenu();
        switch(choice){
            case 1: add(a, b); break;
            case 2: subtract(a, b); break;
            case 3: multiply(a, b); break;
            case 4: divide(a, b); break;
            case 5: cout << "Thoat ...\n"; exit(0);
            default: cout << "Loi: Nhap lai (1-5)\n";
        }
    }while(1);
}

```

```

void add (double a, double b){
    cout << "Tong: " << a + b << endl;
}
void subtract (double a, double b){
    cout << "Hieu: " << a - b << endl;
}
void multiply (double a, double b){
    cout << "Tich: " << a * b << endl;
}
void divide (double a, double b){
    if (b) cout << "Thuong: " << a / b << endl;
    else cout << "Mau bang 0, khong chia duoc\n";
}

```

```

int main(){
    double a, b;
    cout << "Nhap a, b:";
    cin >> a >> b;
    executeMenu(a,b);
    return 0;
}

```


Dùng nguyên mẫu hàm khi viết hàm

- Không dễ để thứ tự định nghĩa hàm phù hợp với thứ tự gọi:
 - Làm được nhưng bất tiện: main phải dưới cùng sau nhiều mã (??)
 - Có lúc không làm được (??)
- Nói lỏng: A được phép gọi B miễn chương trình đã khai nguyên mẫu của B
- Cú pháp:
`<kiểu trả về> <tên hàm> ( <danh sách các tham số> ); // kết thúc là ;`
 - nguyên mẫu giống lời giới thiệu, nên không có phần thân hàm
 - các tham số có thể chỉ khai kiểu, không cần nêu tên

VD: `void print(); bool checkPythagorean (float, float, float)`
- Khai báo mọi nguyên mẫu ở đầu chương trình rồi định nghĩa các hàm sau
=> các hàm đều có thể gọi nhau không cần quan tâm thứ tự.

Thực hành: Viết nguyên mẫu hàm

```
#include <iostream>
using namespace std;
```

```
double calculateRetail(double, double ); // nguyên hàm (prototype)
```

```
int main()
```

```
{
```

```
    double price, retailRate;
```

```
    do{
```

```
        cout << "Gia ban buon: ";
```

```
        cin >> price;
```

```
    }while (price < 0); // còn âm thì còn nhập lại
```

```
    do{
```

```
        cout << "Ty le chi phi ban le: ";
```

```
        cin >> retailRate;
```

```
    }while (retailRate < 0); // còn âm thì còn nhập lại
```

```
    cout << " Gia ban le: " << calculateRetail (price, retailRate) << endl;
```

```
    return 0;
```

```
}
```

```
double calculateRetail(double price , double rate)
```

```
{
```

```
    return price * (1 + rate/100); // return được cả biểu thức
```

```
}
```

Nguyên mẫu chỉ cần khai kiểu tham số, không cần khai tên. C++ hiểu đơn giản đây là hàm nhận 2 tham số double.

Khi định nghĩa thì tham số cần khai kiểu lẫn tên, để sau còn tham chiếu

Thực hành: Viết nguyên mẫu hàm

```
#include <iostream>
using namespace std;

double celcius(double);

int main()
{
    for (int i = 0; i <= 20; i++)
        cout << i << " do F = " << celcius (i) << " do C \n";
    return 0;
}

double celcius(double f)
{
    return 5.0 / 9 * (f - 32); //ép kiểu để có phép chia số thực
}
```

Thực hành: Viết nguyên mẫu hàm

```
#include <iostream>
#include <cmath> // để dùng được hàm pow (...)
using namespace std;

double futureValue (double, double, int);

int main()
{
    double P, i;
    int t;
    cout << "Nhap so tien ban dau, lai ngan hang, so thang gui:";
    cin >> P >> i >> t;
    cout << "So tien se nhan duoc: " << futureValue(P, i, t);
    return 0;
}

double futureValue (double P, double i, int t)
{
    return P * pow(1+ i/100, t); //nếu i là 20 (%) thì lãi suất là 0.2, nên chia cho 100
}
```

Thực hành: Viết nguyên mẫu hàm

Viết chương trình nhập vào 2 số a, b , sau đó hiển thị thực đơn lựa chọn các phép toán: cộng, trừ, nhân, chia và chức năng thoát. Sau khi thực hiện phép toán, chương trình sẽ hiển thị lại thực đơn để người dùng có thể lựa chọn phép toán khác. Quá trình này được lặp lại cho đến khi người dùng chọn thoát

- Phân rã hàm (module hoá)
- Định nghĩa hàm
- Gọi hàm
- Khai nguyên mẫu

```

#include <iostream>
using namespace std;

void executeMenu(double, double);
int printMenu();
void add (double, double);
void subtract (double, double);
void multiply (double, double);
void divide (double, double);

int main(){
    double a, b;
    cout << "Nhap a, b:";
    cin >> a >> b;
    executeMenu(a,b);
    return 0;
}

void executeMenu(double a, double b){
    do {
        int choice = printMenu();
        switch(choice){
            case 1: add(a, b); break;
            case 2: subtract(a, b); break;
            case 3: multiply(a, b); break;
            case 4: divide(a, b); break;
            case 5: cout << "Thoat ...\\n"; exit(0);
            default: cout << "Loi: Nhap lai (1-5)\\n";
        }
    }while(1);
}

```

```

int printMenu(){
    cout << "Nhap thao tac ban muon thuc hien: \\n"
    << "1. Tinh tong \\n"
    << "2. Tinh hieu \\n"
    << "3. Tinh tich\\n"
    << "4. Tinh thuong\\n"
    << "5. Thoat\\n";
    int c;
    cin >> c;
    return c;
}

void add (double a, double b){
    cout << "Tong: " << a + b << endl;
}

void subtract (double a, double b){
    cout << "Hieu: " << a - b << endl;
}

void multiply (double a, double b){
    cout << "Tich: " << a * b << endl;
}

void divide (double a, double b){
    if (b) cout << "Thuong: " << a / b << endl;
    else cout << "Mau bang 0, khong chia duoc\\n";
}

```

Thực hành: Viết nguyên mẫu, định nghĩa, gọi hàm

Quãng đường rơi của vật thể:

Công thức tính quãng đường rơi của vật thể dưới tác dụng của trọng lực là $d = 1/2 gt^2$, trong đó:

d là quãng đường vật thể đã rơi, tính bằng m.

g là trọng lực, bằng 9.8.

t là thời gian rơi, tính bằng giây.

Viết hàm `fallingDistance()` có đầu vào là thời gian rơi, trả ra quãng đường rơi của vật thể.

Viết chương trình dùng vòng lặp in ra màn hình quãng đường rơi theo thời gian từ 1 đến 10 giây.

```
#include <iostream>
```

```
#include <math.h>
```

```
using namespace std;
```

```
const float g = 9.8;
```

```
double fallingDistance(double);
```

```
int main()
```

```
{
```

```
    for (int i=1; i<=10; i++)
```

```
        cout << i << "s: " << fallingDistance(i) << "m \n";
```

```
    return 0;
```

```
}
```

```
double fallingDistance(double t)
```

```
{
```

```
    return g * pow(t, 2)/2;
```

```
}
```

Trả về giá trị

- ❑ Hàm có trả về
- ❑ Hàm trả về void

Hàm có trả về

- Nếu hàm có trả về giá trị thì:
 - phải để kiểu trả về khác void
 - thân hàm phải có lệnh return <giá trị >;
 - <giá trị> này cần khớp kiểu trả về đã khai báo
- <giá trị> khi truyền về caller sẽ dùng như mọi giá trị bình thường:
 - Để gán, để tính toán, để làm điều kiện, để truyền vào hàm, để in ra

```
print();           // không có giá trị trả về thì gọi hàm như một lệnh
sum(m,n);          // có giá trị trả về thì có thể vừa gọi như một lệnh
x = sum(m, n);      // vừa như một giá trị, ở đây là để gán
y = sum(m, n) * 3;   // để tính toán trong biểu thức
z = sum (sqrt(m), sqrt(n)); // để làm đối số cho hàm
if (pythagoreanCheck(a, b, c)) cout << " La 3 canh tam giac vuong\n"; // làm điều kiện
cout << pow(x, y) << endl; // để in ra
```

Thực hành: Hàm có trả về

Số ngày nghỉ

- Viết một chương trình tính số ngày nghỉ trung bình của các nhân viên trong một công ty. Chương trình phải có những hàm sau:
- Một hàm được gọi ở trong hàm main mà yêu cầu người dùng nhập vào số nhân viên của công ty. Hàm này không có tham số, trả về số nguyên dương do người dùng nhập vào.
- Một hàm được gọi trong hàm main, hàm này có một tham số (số nhân viên trong công ty). Hàm này sẽ thực hiện nhập số ngày nghỉ của mỗi nhân viên, sau đó sẽ trả về tổng số ngày nghỉ của các nhân viên.
- Một hàm được gọi trong hàm main, hàm này có hai tham số lần lượt là số nhân viên và tổng số giờ nghỉ của các nhân viên. Hàm này trả về trung bình số ngày nghỉ của các nhân viên trong công ty. Kiểu trả về là double.

Thực hành: Hàm có trả về

```
int nhapNhanVien();
int tongNgayNghỉ(int);
double trungBinhNgayNghỉ (int, int);
int main()
{
    int n = nhapNhanVien();
    int ngay = tongNgayNghỉ(n);
    cout << "Trung binh ngay nghỉ: "
    << trungBinhNgayNghỉ(n, ngay) << endl;
    return 0;
}
```

```
int nhapNhanVien(){
    int n;
    cout << "Số nhân viên: ";
    cin >> n;
    return n;
}
```

```
int tongNgayNghỉ(int n){
    int tong = 0;
    for (int i = 0; i<n; i++){
        int ngay;
        cout << i << ": ";
        cin >> ngay;
        tong += ngay;
    }
    return tong;
}
```

```
double trungBinhNgayNghỉ(int n, int ngay){
    if (ngay) return ngay / double (n);
    else {
        cout << "Không có nhân viên\n";
        return 0;
    }
}
```

Hàm có kiểu trả về là void

- Vẫn dùng được lệnh return:
 - Cú pháp: return ; không có giá trị đi kèm (có thì sẽ báo lỗi)
 - Để chỉ định kết thúc hàm khi ta muốn
- So sánh break, return và exit:
 - break; kết thúc vòng lặp gần nhất, về lệnh ngay sau vòng lặp đó
 - return; kết thúc hàm chứa nó, về lệnh ngay dưới lời gọi hàm đó
 - exit kết thúc chương trình, exit(0) là kết thúc thành công, exit(1) là kết thúc với lỗi
 - Thuộc thư viện `cstdlib`
 - Nếu không nhớ 0 và 1 có thể thay bằng 2 hằng sau:

```
exit(EXIT_SUCCESS);  
exit(EXIT_FAILURE);
```
 - Cuối hàm main vì sao dùng return 0?

Thực hành: Hàm trả về void và lệnh return

Tính tổng

Viết hàm void sum(): hàm sẽ nhập một số n từ bàn phím:

- nếu $n \leq 0$ thì kết thúc luôn hàm
- nếu không thì sẽ nhập vào n số nguyên, tính và in ra tổng của chúng.

```
#include <iostream>
#include <cmath>
using namespace std;
void sum();
int main()
{
    sum();
    return 0;
}
```

```
void sum()
{
    int n;
    cout << "n: ";
    cin >> n;
    if (n<=0) return;
    int tong = 0;
    for (int i = 0; i<n; i++){
        cout << i << " : ";
        int so;
        cin >> so;
        tong += so;
    }
    cout << "Tong: " << tong << endl;
}
```

n âm thì kết hàm!

Thực hành: Trả về giá trị hoặc void

Hình chữ nhật

- Viết chương trình mời nhập chiều rộng và dài của một hình chữ nhật rồi in ra diện tích. Yêu cầu dùng các hàm sau:
- `getLength()`: nhập vào chiều dài và trả lại giá trị đó kiểu `double`.
- `getWidth()`: nhập vào chiều rộng và trả lại giá trị đó kiểu `double`.
- `getArea()`: có 2 tham số là chiều rộng và dài, trả về diện tích của hình chữ nhật.
- `displayData()`: có 3 tham số là chiều rộng, chiều dài và diện tích hình chữ nhật, in ra thông báo chiều rộng, chiều dài và diện tích.

```

double getLength();
double getWidth();
double getArea(double, double);
void displayData(double, double, double);

int main ()
{
    double l, w, area;
    l = getLength();
    w = getWidth();
    area = getArea(l,w);
    displayData(l, w, area);
    return 0;
}

```

```

Chieu dai: 6
Chieu rong: 5
Chieu dai: 6
Chieu rong: 5
Dien tich: 30

```

```

double getLength()
{
    double l;
    cout << "Chieu dai: ";
    cin >> l;
    return l;
}

double getWidth()
{
    double w;
    cout << "Chieu rong: ";
    cin >> w;
    return w;
}

double getArea(double a, double b)
{
    return a * b;
}

void displayData(double a, double b, double c)
{
    cout << "Chieu dai: " << a << endl
        << "Chieu rong: " << b << endl
        << "Dien tich: " << c << endl;
}

```

Truyền tham số cho hàm

- ❑ Tham số và đối số
- ❑ Truyền đối số bằng giá trị
- ❑ Biến tham chiếu
- ❑ Truyền đối số bằng tham chiếu

Tham số và đối số

- Khi định nghĩa hàm, ta coi tham số như một biến tổng quát chưa biết giá trị. Về sau lời gọi hàm mới khởi tạo giá trị cho tham số.
- Quá trình khởi tạo này diễn ra thế nào?

```
void foo (int n)    int n;    foo (3);    int n = 3;
{
    //...
}

foo (a);    int n = a;

foo (x + y);    int n = x + y;

foo (n);    int n = giá trị biến n trong main;
```

- Tham số như vậy chỉ sao chép đối số chứ không phải đối số
 - ❑ n trong foo và a trong main tuy bằng nhau nhưng không phải là một
 - ❑ n trong foo với n trong main cũng vậy

Truyền đối số bằng giá trị

- Ta không thực sự truyền đối số vào hàm, mà chỉ truyền giá trị vào để tham số sao chép. Cơ chế truyền thông tin này vì thế gọi là truyền bằng giá trị (pass by value),
- Hệ quả:
 - Sửa đổi tham số không ảnh hưởng đến đối số (chỉnh sửa ảnh chụp sẽ không ảnh hưởng đến người thật)

```
void foo (int n)
{
    n++;
    cout << "n trong foo: " << n << endl;
}

int n = 7;
cout << "n trong main: " << n << endl;
foo(n);
cout << "n quay ve main: " << n << endl;
```

```
n trong main: 7
n trong foo: 8
n quay ve main: 7
```

Giới thiệu về biến tham chiếu

- Biến: là tên gắn với một ô nhớ được cấp phát ngay khi đó
- Biến tham chiếu: là tên gắn với một ô nhớ được sinh ra từ trước (đích)
- Tham chiếu phải được gắn với đích ngay lúc khai báo (phép khởi gán):
`<kiểu dữ liệu> & <tên tham chiếu> = <đích>;`
`int &r_num = num; // đồng nhất num và r_num`
- Phép khởi gán sẽ đồng nhất tham chiếu và đích, từ đây cả hai là một
 - `num = 3; // r_num bằng 3;`
 - `r_num = 5; // num cũng bằng 5; vì r_num và num chung ô nhớ`
- Phép gán chỉ gán giá trị, không thể gán tham chiếu sang đích khác.
 - `r_num = num1; // chỉ gán giá trị num1 cho r_num, không đồng nhất`
- Đích là một ô nhớ, nên cũng phải là một lvalue => nó chỉ có thể là biến, là một tham chiếu khác, hay là một vài dạng nữa sau này ta sẽ học.

Truyền đối số bằng tham chiếu

Tham số kiểu tham chiếu sẽ thật sự truyền đối số vào hàm

- C++ cho phép khai báo tham số cả ở dạng biến tham chiếu
<kiểu tham số> & <tên tham số>

- Khi đó việc truyền đối số diễn ra thế nào?

```
void foo (int & n)      int & n = ... ;  
{  
    n++;  
    cout << "n trong foo: " << n << endl;  
}
```

```
int n = 5;  
cout << "n trong main: " << n << '\n';  
foo(n);  
cout << "n về lại main: " << n << '\n';
```

foo (3); int & n = 3; error
foo (a); int & n = a;
foo (x + y); int & n = x + y; error
foo (n); int & n = biến n trong main



```
n trong main: 5  
n trong foo: 6  
n về lại main: 6
```

- Cơ chế trên gọi là truyền đối số bằng tham chiếu (pass by reference)

Truyền đối số bằng tham chiếu

- Hệ quả:
 - Tham số chính là đối số, các thay đổi trên tham số cũng xảy ra trên đối số, nên được lưu lại cả khi đã quay về hàm gọi (caller)
- Một hàm có thể có tham số kiểu tham trị lẫn tham chiếu:
 - Nếu cần thay đổi được đối số: chọn truyền bằng tham chiếu
 - Không cần thay đổi đối số: nói chung truyền bằng giá trị (~ sẽ học thêm sau)
 - Biến tham chiếu giúp truyền ngược thông tin về hàm gọi, nên có thể tận dụng như một dạng trả về giá trị, nhờ đó hàm có thể trả về nhiều hơn 1 giá trị.

Ví dụ: Truyền bằng giá trị vs. bằng tham chiếu

// minh hoạ khác biệt giữa truyền bằng giá trị và bằng tham chiếu

```
#include <iostream>
using namespace std;
void incByValue (int);
void incByReference (int &);
int main()
{
    int b = 5;
    cout << " ban dau trong main: " << b << endl ;
    incByValue (b);
    cout << "sau ham incByValue: " << b << endl;
    incByReference (b);
    cout << "sau ham incByReference : " << b << endl;
    return 0;
}

void incByValue (int a)
{
    a++;
    cout << "Trong incByValue: " << a << endl;
}

void incByReference (int & a)
{
    a ++;
    cout << "Trong incByReference: " << a << endl;
}
```

Ví dụ: Truyền bằng giá trị

Đơn vị bán hàng giỏi nhất:

Viết chương trình xác định đơn vị bán hàng giỏi nhất quý của công ty, biết công ty có 4 bộ phận: Đông Bắc, Đông Nam, Tây Bắc và Tây Nam. Chương trình cần có hai hàm sau, được gọi bởi hàm main:

- `double getSales ()` với tham số đầu vào là tên một đơn vị. Nó hỏi doanh số bán hàng trong quý của đơn vị đó, kiểm tra tính hợp lệ (doanh số phải không âm), nếu không hợp lệ thì yêu cầu nhập lại đến khi hợp lệ, sau đó trả về giá trị hợp lệ thu được. Mỗi đơn vị sẽ gọi hàm này một lần.
- `void findHighest ()` với 4 tham số lần lượt là doanh số bán hàng của các đơn vị theo thứ tự liệt kê ở trên. Hàm sẽ in ra tên đơn vị bán hàng giỏi nhất cùng doanh số bán hàng tương ứng.

```
#include <iostream>
using namespace std;
```

```
double getSales(string);
void findHighest(double, double, double, double);
```

```
int main()
{
    double northeast, southeast, northwest, southwest ;
    northeast = getSales("dong bac");
    southeast = getSales("nam bac");
    northwest = getSales("tay bac");
    southwest = getSales("tay nam");

    findHighest (northeast, southeast, northwest, southwest);
    return 0;
}
```

Ví dụ: Truyền bằng giá trị

```
double getSales(string division)
{
    double sales;
    do
    {
        cout << division << ": ";
        cin >> sales;
    }while(sales < 0);
    return sales;
}

void findHighest(double div1, double div2, double div3, double div4)
{
    //lớn nhất nghĩa là lớn hơn mọi cái còn lại
    if (div1 >= div2 && div1 >= div3 && div1 >= div4)
        cout << "Dong Bac ban gioi nhat, doanh so " << div1 << endl;
    if (div2 >= div1 && div2 >= div3 && div2 >= div4)
        cout << "Dong Nam ban gioi nhat, doanh so la " << div2 << endl;
    if(div3 >= div1 && div3 >= div2 && div3 >= div4)
        cout << "Tay Bac ban hang gioi nhat, doanh so la " << div3 << endl;
    if(div4 >= div1 && div4 >= div2 && div4 >= div3)
        cout << "Tay Nam ban gioi nhat, doanh so " << div4 << endl;
}
```


Ví dụ: Truyền kiểu tham chiếu

Loại bỏ điểm thấp nhất:

Viết chương trình tính điểm trung bình điểm các bài kiểm tra đã bỏ đi điểm thấp nhất, gồm:

- hàm void `getScore()` yêu cầu nhập vào một đầu điểm, điểm sẽ lưu trong biến tham chiếu, điểm chỉ thuộc khoảng 0 – 100. Hàm sẽ được gọi nhiều lần từ main để nhập đủ 5 đầu điểm.
- hàm void `calcAverage()` sẽ tính và in ra màn hình điểm trung bình 5 đầu điểm loại đi điểm thấp nhất. Hàm này được gọi một lần ở main.
- hàm int `findLowest()` có tham số là 5 đầu điểm và trả về giá trị nhỏ nhất. Hàm này được gọi trong hàm `calcAverage()`.

```
#include <iostream>
using namespace std;

void getScore (double &);
double findLowest (double, double, double, double, double);
void calcAverage (double, double, double, double, double);

int main()
{
    double mark1, mark2, mark3, mark4, mark5;
    getScore(mark1);
    getScore(mark2);
    getScore(mark3);
    getScore(mark4);
    getScore(mark5);

    //tính trung bình đã loại đi điểm thấp nhất
    calcAverage(mark1, mark2, mark3, mark4, mark5);
    return 0;
}
```

Ví dụ: Truyền kiểu tham chiếu

```
void getScore (double & a)
{
    do {
        cout << "Diem: ";
        cin >> a;
    }while (a < 0 || a > 100); // còn ngoài phạm vi thì nhập lại
}

double findLowest (double a, double b, double c, double d, double e)
{
    double min = a;      //chọn phần tử bất kỳ làm nhỏ nhất sau đó kiểm tra các phần tử còn lại
    if (min > b) min = b;
    if (min > c) min = c;
    if (min > d) min = d;
    if (min > e) min = e;
    return min;
}

void calcAverage (double a, double b, double c, double d, double e)
{
    cout << "Trung binh cua 4 diem cao nhat: " << (a + b + c + d + e - findLowest(a, b, c, d, e)) / 4 << endl;
}
```

Thực hành: Truyền bằng giá trị

Khu vực lái xe an toàn

Một thành phố nọ được chia thành 5 khu vực có tên lần lượt là Đông, Nam, Tây, Bắc và Trung tâm. Viết một chương trình nhập vào số vụ tai nạn giao thông của từng khu vực trong thành phố và in ra khu vực có ít vụ tai nạn nhất.

Chương trình phải có 2 hàm sau:

- `int getNumAccidents()`: đầu vào là tên của khu vực, hàm sẽ yêu cầu người dùng nhập vào số vụ tai nạn của khu vực đó, nếu giá trị nhập vào < 0 thì yêu cầu nhập lại cho đến khi lớn hơn 0. Trong hàm main, mỗi khu vực trong thành phố phải gọi hàm này một lần.
- `void findLowest()`: đầu vào là số vụ tai nạn của 5 khu vực lần lượt theo thứ tự đã nói ở trên

```
#include <iostream>
using namespace std;
```

```
int getNumAccidents(string);
void findLowest(int, int, int, int, int);
```

```

int main()
{
    int east, south, west, north, central;
    east = getNumAccidents("Dong");
    south = getNumAccidents("Nam");
    west = getNumAccidents("Tay");
    north = getNumAccidents("Bac");
    central = getNumAccidents("Trung tam");
    findLowest (east, south, west, north, central);
    return 0;
}

int getNumAccidents(string area)
{
    int num;
    do
    {
        cout << "So vu tai nạn o " << area << " : ";
        cin >> num;
    }while(num < 0);
    return num;
}

void findLowest(int d, int n, int t, int b, int tt)
{
    //nhỏ nhất nghĩa là nhỏ hơn mọi cái còn lại
    if (d <= n && d <= t && d <= b && d <= tt)
        cout << "Dong an toan nhat voi " << d << " vu tai nan \n";
    if (n <= t && n <= b && n <= tt && n <= d)
        cout << "Nam an toan nhat voi " << n << " vu tai nan \n";
    if (t <= n && t <= b && t <= tt && t <= d)
        cout << "Tay an toan nhat voi " << t << " vu tai nan \n";
    if (b <= d && b <= n && b <= t && b <= tt)
        cout << "Bac an toan nhat voi " << b << " vu tai nan \n";
    if (tt <= n && tt <= t && tt <= b && tt <= d)
        cout << "Trung tam an toan nhat voi " << tt << " vu tai nan \n";
}

```

Thực hành: Truyền kiểu tham chiếu

Một cuộc thi tìm kiếm tài năng có 5 giám khảo, mỗi thí sinh sẽ được 5 giám khảo cho điểm từ 0 – 10, điểm có thể lấy 1 số ở phần thập phân, ví dụ điểm 8.3 là hợp lệ. Điểm thí sinh được tính từ trung bình của các giám khảo sau khi loại đi điểm cao nhất và thấp nhất. Viết một chương trình tính điểm điểm thí sinh khi nhập vào điểm của 5 giám khảo, chương trình gồm các hàm sau:

- void getJudgeData() yêu cầu nhập điểm của một giám khảo, điểm sẽ lưu vào một tham số là tham chiếu. Yêu cầu người dùng nhập đến khi điểm hợp lệ thì thôi. Hàm này sẽ được gọi 5 lần ở hàm main để lấy điểm của từng giám khảo.
- void calcScore() tính toán và in ra màn hình điểm 3 giám khảo có được sau khi loại đi điểm cao nhất và thấp nhất. Hàm này được main gọi một lần và truyền vào điểm của 5 giám khảo.

Ngoài ra có hai hàm sau cũng cần tạo và được gọi trong hàm calcScore():

- double findLowest() tìm và trả lại giá trị nhỏ nhất của 5 điểm truyền vào.
- double findHightest() tìm và trả lại giá trị lớn nhất của 5 điểm truyền vào.

```
#include <iostream>
using namespace std;

void getJudgeData (double &);
double findLowest (double, double, double, double, double);
double findHighest (double, double, double, double, double);
void calcScore (double, double, double, double, double);

int main()
{
    double mark1, mark2, mark3, mark4, mark5;
    getJudgeData(mark1);
    getJudgeData(mark2);
    getJudgeData(mark3);
    getJudgeData(mark4);
    getJudgeData(mark5);

    //trung bình các điểm bỏ đi điểm thấp nhất và cao nhất
    calcScore(mark1, mark2, mark3, mark4, mark5);
    return 0;
}
```

```

void getJudgeData (double & a) // tham số kiểu tham chiếu
{
    do {
        cout << "Diem: ";
        cin >> a;
        if (a < 0 || a > 10 || int (a*10)!= a*10)
            break;
    }while (1);
}

double findLowest (double a, double b, double c, double d, double e)
{
    double min;
    min = a; //ban đầu chọn phần tử bất kỳ làm nhỏ nhất
    if (min > b) min = b;
    if (min > c) min = c;
    if (min > d) min = d;
    if (min > e) min = e;
    return min;
}

double findHighest (double a, double b, double c, double d, double e)
{
    double max;
    max = a; // ban đầu chọn phần tử bất kỳ làm lớn nhất
    if (max < b) max = b;
    if (max < c) max = c;
    if (max < d) max = d;
    if (max < e) max = e;
    return max;
}

void calcScore (double a, double b, double c, double d, double e)
{
    //lấy tổng 5 điểm trừ cao nhất và thấp nhất rồi chia 3
    cout << "Diem trung binh cua 3 diem ở giữa là: "
    << (a+b+c+d+e - findHighest(a,b,c,d,e) - findLowest(a,b,c,d,e))/3
    << endl;
}

```

Thực hành: Tham trị vs. Tham chiếu

Viết chương trình nhập vào 2 số a, b , sau đó hiển thị thực đơn lựa chọn các phép toán: cộng, trừ, nhân, chia và chức năng thoát. Sau khi thực hiện phép toán, chương trình sẽ hiển thị lại thực đơn để người dùng có thể lựa chọn phép toán khác. Quá trình này được lặp lại cho đến khi người dùng chọn thoát

- Phân rã hàm (module hoá)
- Định nghĩa hàm
- Gọi hàm
- Khai nguyên mẫu
- Giá trị trả về
- Truyền tham trị vs. tham chiếu


```

void inputA (int & );
void inputB (int & );
void executeMenu (int, int);
void printMenu(int &);
void add(int, int);
void subtract(int, int);
void mul (int, int);
void divide (int, int);

```

```

int main()
{
    int a, b;
    inputA(a);
    inputB(b);
    executeMenu(a, b);
    return 0;
}

```

```

void inputA (int & a)
{
    cout << "a: ";
    cin >> a;
}

```

```

void inputB (int & b)
{
    cout << "b: ";
    cin >> b;
}

```

```

void printMenu(int & choice)
{
    cout << "Moi nhap lua chon (1-5):" << endl
        << "1. Tong" << endl
        << "2. Hieu" << endl
        << "3. Tich" << endl
        << "4. Thuong" << endl
        << "5. Thoat" << endl;
    cin >> choice;
}

```

```

void executeMenu (int a, int b)
{
    do
    {
        int choice;
        printMenu(choice);
        switch(choice)
        {
            case 1: add(a, b); break;
            case 2: subtract(a, b); break;
            case 3: mul (a, b); break;
            case 4: divide (a, b); break;
            case 5: cout << "Thoat ...\n"; exit(0);
            default: cout << "Loi: Nhap lai (1-5) \n";
        }
    }while (1);
}

```

```

void add (int a, int b)
{
    cout << "Tong: " << a + b << endl;
}

```

```

void subtract (int a, int b)
{
    cout << "Hieu: " << a - b << endl;
}

```

```

void mul (int a, int b)
{
    cout << "Tich: " << a * b << endl;
}

```

```

void divide (int a, int b)
{
    if (b) cout << "Thuong: " << float (a) / b << endl;
    else cout << "Loi: Mau bang 0 \n";
}

```

Mở rộng: Chồng hàm

Chồng hàm (Function overloading)

- Là đặt các hàm cùng tên nhưng khác nhau tham số
 - Thường để tạo các hàm làm những nhiệm vụ tương tự

VD:

```
int square (int);           // tính bình phương số int
```

```
float square (float);       // tính bình phương số float
```

- Trình biên dịch phân biệt các lời gọi hàm nhờ chữ ký:
 - Chữ ký = tên hàm và số lượng, kiểu, thứ tự các tham số
 - Nhờ đó, trình biên dịch đảm bảo luôn gọi đúng phiên bản hàm chồng được yêu cầu

Ví dụ: Hàm chồng

- Có các hàm nạp chồng này:

```
void getDimensions(int); // 1
```

```
void getDimensions(int, int); // 2
```

```
void getDimensions(int, double); // 3
```

```
void getDimensions(double, double); // 4
```

- Trình biên dịch sẽ lựa chọn phiên bản ra sao?

```
int length, width;
```

```
double base, height;
```

```
getDimensions(length);
```

```
getDimensions(length, width);
```

```
getDimensions(length, height);
```

```
getDimensions(height, base);
```

fig03_25.cpp
(1 of 2)

Các hàm chồng có cùng tên
nhưng có tham số khác nhau

```
1 // Fig. 3.25: fig03_25.cpp
2 // Using overloaded functions.
3 #include <iostream>
4
5 using std::cout;
6 using std::endl;
7
8 // function square for int values
9 int square( int x )
10 {
11     cout << "Called square with int argument: " << x << endl;
12     return x * x;
13 }
14 // end int version of function square
15
16 // function square for double values
17 double square( double y )
18 {
19     cout << "Called square with double argument: " << y << endl;
20     return y * y;
21 }
22 // end double version of function square
23
```

```
24 int main()
25 {
26     int intResult = square( 7 );           // calls int version
27     double doubleResult = square( 7.5 ); // calls double version
28
29     cout << "\nThe square of integer 7 is " << intResult
30         << "\nThe square of double 7.5 is " << doubleResult
31         << endl;
32
33     return 0; // indicates successful termination
34
35 } // end main
```

fig03_25.cpp
(2 of 2)

fig03_25.cpp
output (1 of 1)

Tùy theo đối số được truyền vào (**int** hoặc **double**) để gọi hàm thích hợp.

Called square with int argument: 7
Called square with double argument: 7.5

The square of integer 7 is 49
The square of double 7.5 is 56.25

Thực hành: Nạp chồng hàm

Tiền nằm viện:

Viết chương trình tính toán chi phí nằm viện. Chương trình sẽ hỏi xem bệnh nhân nhập viện nội trú hay ngoại trú. Nếu nội trú thì cần nhập các thông tin sau:

- Số ngày nằm viện
- Phí nằm viện một ngày
- Chi phí thuốc men
- Phí khám chữa bệnh (xét nghiệm, ...)

Còn nếu bệnh nhân là ngoại trú, chương trình sẽ hỏi các thông tin sau:

- Phí khám chữa bệnh (xét nghiệm, ...)
- Chi phí thuốc men

Chương trình cần 2 hàm chồng để tính chi phí: một hàm 4 tham số tính chi phí bệnh nhân nội trú và một hàm 2 tham số tính chi phí bệnh nhân ngoại trú. 2 hàm đều trả về số tiền phải trả.

```
double hospitalFee(double test, double medical)
{
    return test + medical;
}
```

```
double hospitalFee(double test, double medical, double price, int days)
{
    return test + medical + days * price;
}
```

Số lượng tham số ở đây khác nhau, dù có lúc giống số lượng mà khác kiểu vẫn tính là 2 danh sách tham số khác nhau

Thực hành: Nạp chồng hàm

```
#include <iostream>
#include <fstream>
using namespace std;
```

```
double hospitalFee(double, double);
double hospitalFee(double, double, double, int);
```

```
int main()
{
    bool in;
    double test, medical, price;
    int days;
    cout << "Noi tru = 1, ngoai tru = 0 : ";
    cin >> in;

    cout << "Phi xet nghiem và phi thuốc men: ";
    cin >> test >> medical;
    if (in){
        cout << "Gia nam vien mot ngay và số ngày nằm viện: ";
        cin >> price >> days;
        cout << "Tong chi phi: " << hospitalFee(test, medical, price, days) << endl;
    }
    else cout << "Tong chi phi: " << hospitalFee(test, medical) << endl;
    return 0;
}
```

Dựa vào số lượng và kiểu các đối số, trình biên dịch sẽ đoán được đang gọi hàm chồng nào

Mở rộng: Đối số mặc định

Đối số mặc định

- Là giá trị tự động truyền cho tham số nếu lời gọi hàm thiếu đối số đó

```
void evenOrOdd(int = 0);           // nếu có nguyên mẫu thì để đối số mặc định
void evenOrOdd(int a){...}         // ở nguyên mẫu còn bỏ ở tiêu đề
float square(float a = 0){...}     // chỉ để ở tiêu đề nếu không có nguyên mẫu
```
- Thiết lập được đối số mặc định cho nhiều tham số:

```
int getSum(int, int t= 0, int = 0);
```
- Lời gọi hàm lúc này được phép bỏ qua các đối số đã khai mặc định:

```
getSum(a);           // tương đương getSum(??);
getSum(a, b);        // tương đương getSum(??);
getSum(a, b, c);     // 3 lời gọi đều OK
```

 - Từ trái sang phải, lấy dần đối số khớp cho tham số, đến đâu thiếu thì xem liệu có đối số mặc định, có thì khớp vào, không có thì báo lỗi

Ví dụ: Đối số mặc định

Program 6-23

```
1 // This program demonstrates default function arguments.
2 #include <iostream>
3 using namespace std;
4
5 // Function prototype with default arguments
6 void displayStars(int = 10, int = 1);
7
8 int main()
9 {
10     displayStars();           // Use default arguments
11     cout << endl;
12     displayStars(5);          // Use default argument for rows
13     cout << endl;
14     displayStars(7, 3);       // Use 7 columns and 3 rows
15     return 0;
16 }
```

Đối số mặc định khai báo trong nguyên mẫu

```
18 //*****
19 // Definition of function displayStars.
20 // The default argument for cols is 10 and for rows is 1.
21 // This function displays a square made of asterisks.
22 //*****
23
24 void displayStars(int cols, int rows)
25 {
26     // Nested loop. The outer loop controls the rows
27     // and the inner loop controls the columns.
28     for (int down = 0; down < rows; down++)
29     {
30         for (int across = 0; across < cols; across++)
31             cout << "*";
32         cout << endl;
33     }
34 }
```

Program Output

Lưu ý với đối số mặc định

- Đối số mặc định có thể là hằng, biến toàn cục, hay lời gọi hàm
`float squareRoot(float a = b);` // OK nếu b là toàn cục và đã định nghĩa
- Tham số có đối số mặc định không được ở trước tham số không có đối số mặc định:
`int getSum(int, int = 0, int = 0);` // OK
`int getSum(int, int = 0, int);` // báo lỗi
- Đã bỏ qua đối số nào, mọi đối số sau đó cũng phải bỏ
`sum = getSum(num1, num2);` // OK
`sum = getSum(num1, , num3);` // báo lỗi

Tổng kết về hàm

- Khái niệm lập trình có cấu trúc: chia mã thành nhiều hàm con
- Cách khai báo nguyên mẫu, định nghĩa hàm, gọi hàm
- Truyền đối số kiểu giá trị vs. kiểu tham chiếu
- Các cách sử dụng giá trị trả về của hàm
- Mở rộng: hàm chồng, đối số mặc định

Trò chơi đoán số (tiếp)

Máy tính nghĩ ra một số ngẫu nhiên trong khoảng từ 1 đến 1000, nó giữ bí mật và mời bạn đoán. Mỗi lần đoán, nó sẽ cho bạn thắng nếu bạn trả lời đúng số cần đoán. Nếu không đúng, bạn được biết giá trị vừa đoán là quá cao hay quá thấp so với số cần đoán. Sau 10 đoán mà vẫn không đoán trúng thì bạn sẽ thua.

Hello! What is your name?

Mai

Play with me, Mai! I am thinking of a number between 1 and 1000.

Take a guess.

10

Your guess is too high.

Take a guess.

2

Your guess is too low.

Take a guess.

4

Well done, Mai! You guessed my number in 3 guesses!

Trò chơi đoán số - Tuần 5

- Chương trình trò chơi đoán số của ta đã chạy được từ tuần trước, nhưng giờ ta muốn có một giao diện đẹp hơn và mã chương trình tổ chức gọn gàng hơn
- Tuần này hãy mở rộng chương trình như sau:
 - Bắt đầu trò chơi, hãy hiển thị giao diện thực đơn:
 - Đoán số ngẫu nhiên từ 1 – 1000
 - Đoán số ngẫu nhiên từ 1 đến n với n do người dùng chọn
 - Thoát
 - Nếu người dùng chọn thoát thì thoát, nếu không thì thực hiện tính năng tương ứng (một lần chơi), chơi xong thì lại hiển thị thực đơn tiếp.
 - Hãy tổ chức chương trình bằng cách chia thành các hàm con một cách hợp lý (vừa đủ hàm, không quá ít hay quá nhiều)